

 AGENT MEMORY

The Memory Stack Behind AI Agents

Memory is why your agent feels intelligent
- or doesn't.

 7 Memory Layers

 Python Code

 Production Ready



AI Coach John

FOLLOW



PROITBRIDGE
Simpler For Smarter Futures

 Repost

What Is Agent Memory & Why It Matters

Unlike human memory which happens naturally, AI memory must be deliberately engineered. It requires complex systems to decide exactly what to store, how to structure it, and when to retrieve it into the context window.

💡 THE CORE PROBLEM

An LLM has **no memory by default**. Every call starts from zero. Without memory, your agent can't learn from past interactions, maintain context across sessions, avoid repeating mistakes, or chain long multi-session tasks. It remains stuck as a stateless function rather than an intelligent assistant.

✖ WITHOUT MEMORY

- Session 1: User explains situation
- Session 2: User re-explains again
- Session 3: User re-explains again
- Session 4: User gives up

"Hi, how can I help?" - every time

✔ WITH MEMORY

- Session 1: Agent learns preferences
- Session 2: Agent remembers + builds
- Session 3: Agent anticipates needs
- Session 4: Task completed

"Welcome back Sarah, continuing from last week..."

★ **Memory is what makes an agent feel like a colleague, not a search engine.**



AI Coach John

FOLLOW



↻ Repost

The 7-Layer Memory Stack

Memory is not a single database. It is a multi-tiered architecture that balances retrieval speed, token cost, and relevance. A robust agent uses the right memory layer for the right task to optimize both performance and cost.

LAYER	STORAGE	DURATION
 Context Window	In-model	This call
 Session Store	Redis / RAM	This session
 Short-term	PostgreSQL	Days-weeks
 Episodic	Vector DB	Weeks-months
 Semantic	Vector DB	Permanent
 Procedural	Prompt / DB	Permanent
 Long-term	Snowflake/BQ	Permanent

- Each layer answers a different question: "What now?" → context. "Last week?" → episodic. "What do I know?" → semantic.



Context Window - What's Happening Now

📌 WHAT IT IS

The active content the LLM sees right now. Capacity: **8K–200K+ tokens** depending on model. Lives here: current conversation, retrieved docs, tool results, system prompt.

```
from anthropic import Anthropic
client = Anthropic()

class ContextWindowManager:
    def __init__(self, max_tokens: int = 180_000):
        self.messages = []
        self.max_tokens = max_tokens

    def add_message(self, role: str, content: str):
        self.messages.append({"role": role, "content": content})
        # Prune if approaching limit
        if self.estimate_tokens() > self.max_tokens * 0.85:
            self.summarize_and_compress()

    def summarize_and_compress(self):
        early = self.messages[:-6]
        recent = self.messages[-6:]
        summary = self.summarize(early)
        self.messages = [
            {"role": "system",
             "content": f"Summary: {summary}"}
        ] + recent
```

✓ BEST FOR

In-conversation context. Follow-up questions. Tool result chaining.

⚠️ LIMITATION

Disappears when session ends. Expensive at 200K tokens.



Session Memory - This Conversation's History

📌 WHAT IT IS

Persistent storage of the current session – survives page refreshes. Stores: full message history, task states, user preferences, tool results worth referencing later.

```
import redis, json

class SessionMemory:
    def __init__(self, session_id: str, ttl: int = 7200):
        self.redis = redis.Redis()
        self.session_id = session_id
        self.ttl = ttl # 2 hours

    def save_message(self, role: str, content: str):
        key = f"session:{self.session_id}:messages"
        msg = json.dumps({"role": role, "content": content})
        self.redis.rpush(key, msg)
        self.redis.expire(key, self.ttl)

    def get_history(self) → list:
        key = f"session:{self.session_id}:messages"
        raw = self.redis.lrange(key, 0, -1)
        return [json.loads(m) for m in raw]

    def save_state(self, key: str, value: dict):
        sk = f"session:{self.session_id}:state:{key}"
        self.redis.setex(sk, self.ttl, json.dumps(value))
```



Storage options: Redis (fastest) · DynamoDB (scalable) · PostgreSQL (queryable)



AI Coach John

FOLLOW



🔄 Repost

Short-Term Memory - Recent Sessions (Days-Weeks)

🔧 WHAT IT IS

Compressed summaries of recent past sessions - loaded at the start of new sessions to restore context without full history replay.

Session 1 ends → **Summarizer compresses conversation**



```
{ "user_id": "sarah_k", "summary": "Discussed Enterprise plan for 50 users.
Prefers annual billing. Needs SSO.", "open_tasks": ["send_pricing_doc"] }
```



Session 2 begins → **Summary loaded** → **Agent has full context**

```
def create_session_summary(messages: list, user_id: str) → dict:
    summary = llm.generate(f"""
    Summarize: key decisions, preferences,
    tasks completed/pending, facts to remember.
    Conversation: {messages}
    """)
    return {
        "user_id": user_id,
        "session_date": datetime.now().isoformat(),
        "summary": summary,
        "action_items": extract_action_items(summary)
    }
```



Episodic Memory - Remembering Specific Events

🧠 HUMAN ANALOGY

"I remember the last time we tried this approach, it failed because of X." The agent recalls that specific episode when facing a similar situation.

```
class EpisodicMemory:
    def __init__(self, vector_store):
        self.store = vector_store

    def store_episode(self, episode: dict):
        text = f"""
        Date: {episode['date']}
        Task: {episode['task']}
        Approach: {episode['approach']}
        Outcome: {episode['outcome']}
        Lesson: {episode['lesson']}
        """
        self.store.add_texts([text], metadatas=[episode])

    def recall_similar(self, situation: str, k=3) → list:
        return [doc.metadata for doc in
                self.store.similarity_search(situation, k=k)]

# Example episode
episode = {
    "date": "2026-06-10",
    "task": "Research competitor pricing",
    "approach": "Web search 'pricing' keyword",
    "outcome": "Only marketing pages returned",
    "lesson": "Use pricing table on brand keyword"
```



Store: past tool outcomes, user feedback, task results, exception cases.



AI Coach John

FOLLOW



PROTIBRIDGE
Bridging The Safety Gap

🔄 Repost

Semantic Memory - What the Agent Knows

📌 WHAT IT IS

Accumulated factual knowledge - facts, concepts, domain knowledge. **Episodic** = "this happened to me." **Semantic** = "I know that X is true."

🗂️ CATEGORIES

- User profiles & preferences
- Domain-specific facts
- Product knowledge
- Learned rules & exceptions

</> CODE

```
class SemanticMemory:
    def __init__(self, store, llm):
        self.store = store
        self.llm = llm

    def extract_facts(self, conv: list):
        facts = self.llm.generate(f"""
        Extract durable facts only.
        Ignore temp context.
        Conversation: {conv}
        Return JSON list.
        """)
        for fact in json.loads(facts):
            self.store.add_texts([fact])

    def recall(self, query: str):
        return self.store.similarity_search(
            query, k=5
        )
```



Example: "Sarah prefers brief summaries, dislikes jargon" - stored once, used across every session forever.



AI Coach John

FOLLOW



🔄 Repost

Procedural Memory - How to Do Things

👤 HUMAN ANALOGY

A new employee reads the onboarding handbook. A tenured employee just knows how things work. Procedural memory gives agents that institutional knowledge of *how* to execute recurring tasks.

```
PROCEDURAL_MEMORY = {
    "handle_refund_request": [
        "1. Verify order exists in system",
        "2. Check if within 30-day return window",
        "3. If yes: initiate_refund(order_id, amount)",
        "4. If no: escalate_to_human('outside_window')",
        "5. Send confirmation email"
    ],
    "research_competitor": [
        "1. Search '[competitor] pricing plans 2024'",
        "2. Fallback: '[competitor] plans site:techcrunch.com'",
        "3. Extract: names, prices, features, limits",
        "4. Query internal DB for same fields",
        "5. Build comparison table"
    ]
}

def load_procedure(task_type: str) → list:
    best = procedure_db.similarity_search(task_type, k=1)
    return PROCEDURAL_MEMORY.get(
        best[0].metadata["key"], []
    )
```

💡 Store: SOPs, tool usage patterns, prompt templates, workflow sequences.



Long-Term Storage - The Agent's Permanent Record

WHAT IT IS

Persistent archive of **all** agent interactions. Too large to load into context. Used for: analytics, compliance audits, retraining, and identifying failure patterns.

Agent interactions → Event Stream (Kafka / SQS)



DATA WAREHOUSE (Snowflake / BigQuery)

• Interaction logs • Tool call records • Outcomes + scores



Analytics: "What % of tasks fail at step 3?"

✓ USE FOR

Analytics, compliance, GDPR audit trails, retraining datasets.

✗ NEVER STORE

Raw PII, unmasked data, every tool result verbatim (too noisy).



AI Coach John

FOLLOW



 Repost

4 Ways to Retrieve the Right Memory

🕒 Recency-based

🔍 Relevance-based

★ Importance-based

📊 Hybrid (best)

📅 HYBRID SCORING FORMULA

$$\text{score} = (\alpha \times \text{semantic_similarity}) + (\beta \times \text{recency_score}) + (\gamma \times \text{importance_score})$$

 $\alpha + \beta + \gamma = 1.0$ Example: $\alpha=0.5, \beta=0.3, \gamma=0.2$

```
def retrieve_memories(query: str, user_id: str, k=5):
    hits = vector_store.similarity_search_with_score(
        query, k=k*3, filter={"user_id": user_id}
    )
    scored = []
    for doc, sem_score in hits:
        age_h = (datetime.now() -
                 doc.metadata["timestamp"]).total_seconds() / 3600
        recency = 1 / (1 + age_h / 24)
        importance = doc.metadata.get("importance", 0.5)
        final = 0.5*sem_score + 0.3*recency + 0.2*importance
        scored.append((doc, final))
    return [d for d, _ in sorted(scored, key=lambda x: -x[1][:k])]
```



Full Production Memory Stack

MEMORY LAYER	RECOMMENDED	ALTERNATIVE
Context Window	In-model context	-
Session	Redis (<2hr TTL)	DynamoDB
Short-term	PostgreSQL + pgvector	Chroma
Episodic	Pinecone / Weaviate	Qdrant
Semantic	Pinecone / Weaviate	pgvector
Procedural	Prompt template DB	YAML files
Long-term	Snowflake / BigQuery	S3 + Athena

⚠ NEVER STORE IN MEMORY

- > Raw PII - mask before storing
- > Every tool result verbatim - too noisy
- > Failed interactions without cleanup
- > Outdated facts - implement TTL or versioning



AI Coach John

FOLLOW



 Repost

Memory Mistakes That Break Production Agents

Even with the perfect stack, poor implementation will cause your agent to hallucinate, run up massive API bills, or fail entirely. These are the most common pitfalls engineering teams hit when moving from prototype to production.

- ❗ **No memory pruning** - Context window fills up → agent fails mid-task. Always implement token budget management.
- ❗ **Storing everything** - 100% of interactions creates noise. Only store important, actionable facts.
- ❗ **No TTL on session data** - Stale sessions accumulate in Redis. Set TTLs on all session stores.
- ❗ **PII in memory stores** - Raw user data violates GDPR/HIPAA. Always mask before storing.
- ❗ **Single memory layer** - Context window only → no cross-session continuity. Long-term only → slow retrieval. Use all layers.
- ❗ **No conflict handling** - Two contradicting memories confuse the agent. Newer fact must override older one with timestamp tracking.



Agent Memory Stack Cheatsheet

LAYER	WHAT IT STORES	DURATION	STORAGE
Context Window	Active conversation + docs	This call	In-model
Session Memory	Full conversation this session	Hours	Redis
Short-term	Compressed session summaries	Days-weeks	PostgreSQL
Episodic	Specific past events + outcomes	Weeks-months	Vector DB
Semantic	Facts, preferences, knowledge	Permanent	Vector DB
Procedural	How-to workflows, playbooks	Permanent	DB / Prompts
Long-term	All interactions (analytics)	Permanent	Data Warehouse

🕒 Match memory layer to how far back you need to go

This call → Context · Hours → Session · Weeks → Episodic · Always → Semantic



AI Coach John

FOLLOW



PROITBRIDGE
Smart For Smarter Futures

↻ Repost